Lecture 3 - TCP Congestion Control

Congestion collapse occurs when a network carries so many retransmitted packets that useful throughput approaches zero. It was first observed on the NSFNET backbone in 1986, when throughput between two sites dropped from 32 kbit/s to 40 bit/s.

The sender maintains a congestion window, written cwnd, which limits how many unacknowledged bytes may be in flight. The effective send window is the minimum of cwnd and the receiver's advertised window. TCP has no explicit signal from the network, so it infers congestion from packet loss.

Slow start begins with cwnd set to one maximum segment size. Every acknowledgement increases cwnd by one MSS, which doubles the window each round-trip time. Despite the name, slow start is exponential growth; it is slow only in comparison to immediately transmitting a full receiver window.

Lecture 3 - Congestion Avoidance and Fast Recovery

When cwnd reaches the slow start threshold, ssthresh, TCP switches to congestion avoidance. In this phase cwnd grows by one MSS per round-trip time rather than per acknowledgement, giving linear growth. This is the additive increase half of AIMD.

On a timeout, ssthresh is set to half the current cwnd and cwnd is reset to one MSS, returning the connection to slow start. This is the multiplicative decrease half of AIMD. The combination of additive increase and multiplicative decrease is what allows competing flows to converge on a fair share of a bottleneck link.

Three duplicate acknowledgements are treated as a weaker congestion signal than a timeout, because they prove later packets are still arriving. Fast retransmit resends the missing segment immediately without waiting for the retransmission timer. Fast recovery then halves cwnd instead of dropping to one MSS, which is the key difference between TCP Tahoe and TCP Reno.

Lecture 3 - Modern Variants

CUBIC, the default on Linux since kernel 2.6.19, replaces linear growth with a cubic function of the time since the last congestion event. This makes the window growth independent of the round-trip time, which fixes the unfairness that Reno shows when flows with very different RTTs share a link.

BBR takes a different approach entirely. Instead of treating loss as the congestion signal, it builds a model of the bottleneck bandwidth and the minimum round-trip time, then paces packets to match. This avoids the bufferbloat problem, where large router buffers absorb the loss that loss-based algorithms depend on and latency grows without bound.